Source code information:

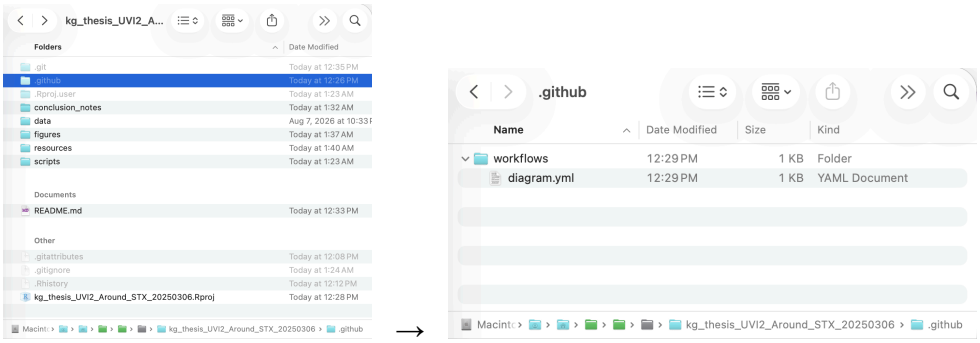
Visualizing a Codebase

https://github.com/githubocto/repo-visualizer

https://github.com/githubocto/repo-visualizer-demo

On Mac, hit `Shift` ⌘ `.` to show hidden files (including folders/files that start with "." like ".github)

1️⃣ In local repo folder, create a new folder named **`.github`** → create subfolder inside `.github` named **`workflows`** → inside `workflows` create a new .txt file and save it as **`diagram.yml`**



2️⃣ Copy paste the code block below into the **`diagram.yml`** file **keep all indentation/spacing exactly as is*

```
name: Create repo diagram

# Run only manually, from the Actions tab (to update diagram: Actions tab → click on "Create repo diagram" workflow in the left sidebar → click "Run workflow" dropdown → ensure "main" branch is selected → click "Run workflow" button)
on:
  workflow_dispatch:

# The action needs write access to push the diagram commit
permissions:
  contents: write

jobs:
  diagram:
    runs-on: ubuntu-latest
    steps:

      # Pull a copy of the repo onto GitHub's runner
      - name: Checkout code
        uses: actions/checkout@v4

      # Build the SVG only; pushing is done in the next step because the action's own push fails once the "diagram" branch exists (the action is downloaded and run directly instead of with "uses:" so its folder-label rules can be changed first; settings are passed as INPUT_ variables, which is how "with:" values reach the action)
      - name: Update diagram
        env:
          INPUT_OUTPUT_FILE: "diagram.svg"
          INPUT_SHOULD_PUSH: "false"

          # Folders to exclude from diagram
          INPUT_EXCLUDED_PATHS: ".git,.github,.Rproj.user,tree"

          # Files to exclude from diagram (**/.DS_Store = all .DS_Store files in all subfolders; **/java_maxent = all java_maxent folders and their contents; **/erddap_raw/*.nc = all .nc files INSIDE 'erddap_raw' folder)
          INPUT_EXCLUDED_GLOBS: ".gitattributes;.gitignore;**/.DS_Store;**/java_maxent;**/erddap_raw/*.nc"

          # Folders that always get a label, even if their circle is small (full path from repo root, separated by semicolons)
          LABEL_SHOW: "scripts/2_interpolation_scripts;scripts/4_analysis_scripts/maxent_scripts;data/3_data_analyzed;data/4_data_interpolated"

          # Folders that never get a label (full path from repo root, separated by semicolons)
          LABEL_HIDE:
"scripts/1A_acoustic_scripts/HF_scripts;scripts/1A_acoustic_scripts/LF_scripts;data/1_data_raw/acoustic_raw;data/2_data_processed/acoustic_processed/LF_processed;data/3_data_analyzed/acoustic_analyzed/LF_analyzed;data/3_data_analyzed/acoustic_analyzed/HF_analyzed;data/5_data_gis/LF_gis_data;data/5_data_gis/HF_gis_data;data/6_data_analysis_outputs/maxent_model/3_HF_maxent_output/odontocetes;data/6_data_analysis_outputs/maxent_model/3_HF_maxent_output/sperm;data/6_data_analysis_outputs/maxent_model/3_HF_maxent_output/uo;data/6_data_analysis_outputs/maxent_model/3_LF_maxent_output/humpback;data/6_data_analysis_outputs/maxent_model/3_LF_maxent_output/minke"

          # Custom colors per file extension (anything not listed keeps its default color)
          INPUT_FILE_COLORS: '{"csv": "#ff00f2", "png": "#91c8ff", "py": "#00F6FF", "ipynb": "#ae9fff", "tif": "#5900ff", "pdf": "#8ef6ce", "qmd": "#2DA1FF", "rds": "#DDA0DD", "txt": "#FFC300", "nc": "#24267d", "data": "#FFF68F"}'

        run: |
          # Download version 0.7.1 of the action into a temporary folder on the runner
          git clone --quiet --depth 1 --branch 0.7.1 https://github.com/githubocto/repo-visualizer.git "$RUNNER_TEMP/repo-visualizer"

          # Write a small script that changes the action's label rules so LABEL_SHOW folders always get a label and LABEL_HIDE folders never do (it also adds INPUT_FILE_COLORS to the action's color list, because version 0.7.1 has no file_colors setting of its own)
          cat > "$RUNNER_TEMP/patch-labels.js" <<'EOF'
          const fs = require("fs");
          const file = process.argv[2];
          let src = fs.readFileSync(file, "utf8");
          const swaps = [
            ["if (r4 < 16 && selectedNodeId !== data2.path)",
             'const findLabelRule = (envName) => (process.env[envName] || "").split(";").map((s) => s.trim()).filter(Boolean).find((t) => data2.path === t.slice(1) || (data2.path.startsWith(t.slice(1) + "/") && data2.label.startsWith(t.split("/").pop() + "/")));\n    if (findLabelRule("LABEL_HIDE"))\n      return null;\n    const showRule = findLabelRule("LABEL_SHOW");\n    if (!showRule && r4 < 16 && selectedNodeId !== data2.path)'],
            ["if (data2.label.length > r4 * 0.5)",
             "if (!showRule && data2.label.length > r4 * 0.5)"],
            ["const label = truncateString(data2.label,",
```

```
            'const label = showRule ? showRule.split("/").pop() : truncateString(data2.label,'],
          ["let offsetR = r4 + 12 - depth * 4;",
           "let offsetR = r4 + 12 - depth * 4;\n    if (showRule)\n      offsetR = Math.max(offsetR, label.length * (16 - depth) * 0.62 / (2 * Math.PI) + 3);"],
        ];
        for (const [oldText, newText] of swaps) {
          if (src.split(oldText).length !== 2) {
            console.error("Could not find label code to change: " + oldText);
            process.exit(1);
          }
          src = src.replace(oldText, newText);
        }
        const colorStart = src.indexOf("var language_colors_default = {");
        const colorEnd = src.indexOf("\n};", colorStart);
        if (colorStart === -1 || colorEnd === -1) {
          console.error("Could not find color list to change");
          process.exit(1);
        }
        src = src.slice(0, colorEnd + 3) + '\nObject.assign(language_colors_default, JSON.parse(process.env.INPUT_FILE_COLORS || "{}"));' + src.slice(colorEnd + 3);
        fs.writeFileSync(file, src);
        console.log("Label rules and colors changed");
        EOF

        # Apply the label change, then run the action to build diagram.svg
        node "$RUNNER_TEMP/patch-labels.js" "$RUNNER_TEMP/repo-visualizer/index.js"
        node "$RUNNER_TEMP/repo-visualizer/index.js"

    # Replace the "diagram" branch with a single commit holding only the new diagram.svg (main is not touched)
    - name: Push diagram to diagram branch
      run: |
        git config user.name "github-actions[bot]"
        git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
        mv diagram.svg "$RUNNER_TEMP/diagram.svg"
        git switch --orphan diagram
        mv "$RUNNER_TEMP/diagram.svg" diagram.svg
        git add diagram.svg
        git commit -m "Repo visualizer: updated diagram"
        git push --force origin diagram
```

3) Add the following line to the repo's **`README.md`** file **change the repo name in the path to fit current repo*

```
![Visualization of the codebase](https://github.com/keileygregory/*repo-name*/blob/diagram/diagram.svg?raw=true)
```

4) Open repo R project in R Studio, commit **ONLY** the **`diagram.yml`** file + the **`README.md`** file, and **push** as normal

5) Go to online repo webpage in GitHub → go to **`Actions`** tab in top menu bar of repo webpage → click on "**`Create repo diagram`**" workflow in the left sidebar → click `Run workflow` button to in top right corner to open dropdown menu → ensure `main` branch is selected *(this is the branch containing the files you want shown in the diagram and the current version of the diagram.yml file itself, NOT where the diagram gets saved to)* → click **`Run workflow`** button!

